SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

Symbiosis Institute of Technology, Pune

Department of Artificial Intelligence and Machine Learning

EXPERIENTIAL LEARNING · LABORATORY / COURSE REPORT

Unstructured Data into a NoSQL Database

Big Data Analytics — MongoDB on Ubuntu 24.04

STUDENT DETAILS (for official submission)

NAME	**Sukhada Dhingra**
PRN	**24070126505**
BATCH	**A3**
PROGRAMME	**B.Tech. AIML**
COURSE / PAPER	**Big Data Analytics**

1. Certificate / Declaration

I, Sukhada Dhingra, PRN 24070126505, Batch A3, student of B.Tech. (Artificial Intelligence and Machine Learning) at Symbiosis Institute of Technology, Pune, constituent of Symbiosis International (Deemed University), hereby declare that the work presented in this laboratory report titled “Unstructured Data into a NoSQL Database” is my original work carried out as part of the course Big Data Analytics. MongoDB Community Server 8.0.32 was installed and run on Ubuntu 24.04.4 LTS. Three hundred unstructured JSON review documents were imported with `mongoimport`, and CRUD plus aggregation queries were executed in `mongosh`. Screenshots attached in this PDF are captured from that execution and have not been copied from another student.

I further declare that standard references (MongoDB documentation, Sadalage & Fowler NoSQL Distilled, course notes on CAP and document stores) are cited wherever theory is summarised, and that no part of this report has been submitted elsewhere for academic credit.

2. Abstract

This laboratory exercise stores unstructured / semi-structured product-review JSON in a document NoSQL database (MongoDB) on Ubuntu. Relational tables assume a fixed schema; review dumps do not: nested product and customer objects, optional titles, tag arrays, media blobs, seller replies and ETL leftovers appear on some records and not others. MongoDB accepts each record as a BSON document without ALTER TABLE. A generator wrote 300 JSONL documents; `mongoimport` loaded them into `sit_bda.product_reviews` (300 imported, 0 failed). mongosh then demonstrated CRUD (count went 300 → 301 → 300), projection, sort/skip/limit, and aggregation ($group by category, $match on Pune, $unwind on tags, $facet histogram). Electronics led volume (81 reviews, mean rating 3.84). Optional fields prove the schema-less claim: tags 79%, title 72%, media 20%, seller_reply 10%, return_reason 2%.

3. Contents

§	Section
1	Certificate / Declaration
2	Abstract
3	Contents
4	Aim, objectives and apparatus
5	Unstructured data and NoSQL
6	MongoDB document model and CAP
7	Dataset and schema-less design
8	Implementation procedure (Ubuntu)
9	Source listings
10	Execution screenshots
11	Results and analysis

§	Section
12	Observations, viva notes and limitations
13	Conclusion
14	References

4. Aim, objectives and apparatus

4.1 Aim

To ingest unstructured JSON into a NoSQL (MongoDB) database on Ubuntu, to design a loose document schema that tolerates missing and nested fields, and to run CRUD plus aggregation queries that would be awkward in a rigid relational schema.

4.2 Objectives

- Install and start MongoDB Community Server on Ubuntu 24.04 and verify mongod / mongosh / mongoimport.
- Generate an unstructured product-review corpus (JSON Lines) with nested objects and optional fields.
- Create a database and collection with a *loose* JSON Schema (required keys only; additionalProperties true).
- Bulk-load the corpus with mongoimport and confirm document counts.
- Perform insert, find (filter + projection), update and delete on the collection.
- Run aggregations: $group, $match, $unwind, $sort / $skip / $limit, and $facet.

4.3 Apparatus (actually used)

Student	Sukhada Dhingra · PRN 24070126505 · Batch A3
OS	Ubuntu 24.04.4 LTS (Linux 6.12.94+ x86_64)
NoSQL engine	MongoDB Community Server 8.0.32 (mongod, WiredTiger)
Shell / tools	mongosh 2.10.0 · mongoimport 100.18.0
Language	Python 3.12 (dataset generator) · mongosh JavaScript (queries)
Database	sit_bda.product_reviews · localhost:27017
Corpus	300 JSONL product-review documents (schema-less)

5. Unstructured data and NoSQL

Big Data is often described by volume, velocity and variety. Variety is the reason this lab exists. Clickstream logs, product reviews, social posts and device telemetry rarely arrive as tidy 3NF tables. They arrive as JSON, XML, free text and nested maps whose keys change from event to event. Forcing that feed into VARCHAR columns either drops fields or explodes the schema. NoSQL document stores keep the native shape: one record ≈ one JSON document, arrays and sub-documents allowed, missing keys allowed.

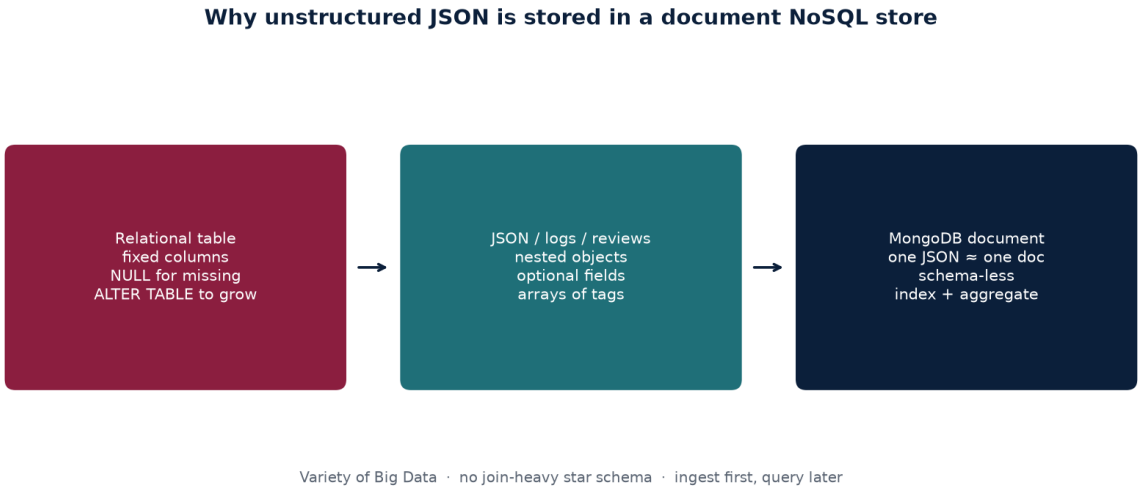


Figure 1. Unstructured JSON maps onto MongoDB documents; a rigid table does not.

The four common NoSQL families are key–value, document, wide-column and graph. This lab uses the document family (MongoDB). It is the usual choice when the payload is already JSON and the queries need secondary indexes plus aggregation, not just get-by-key.

6. MongoDB document model and CAP

6.1 Logical stack

A MongoDB deployment is one or more `mongod` processes. Data sits in databases; databases hold collections; collections hold documents stored as BSON (Binary JSON). There is no JOIN-centric schema. Related data is embedded (product inside the review) or referenced by id. This lab embeds product and customer because they travel with the review.

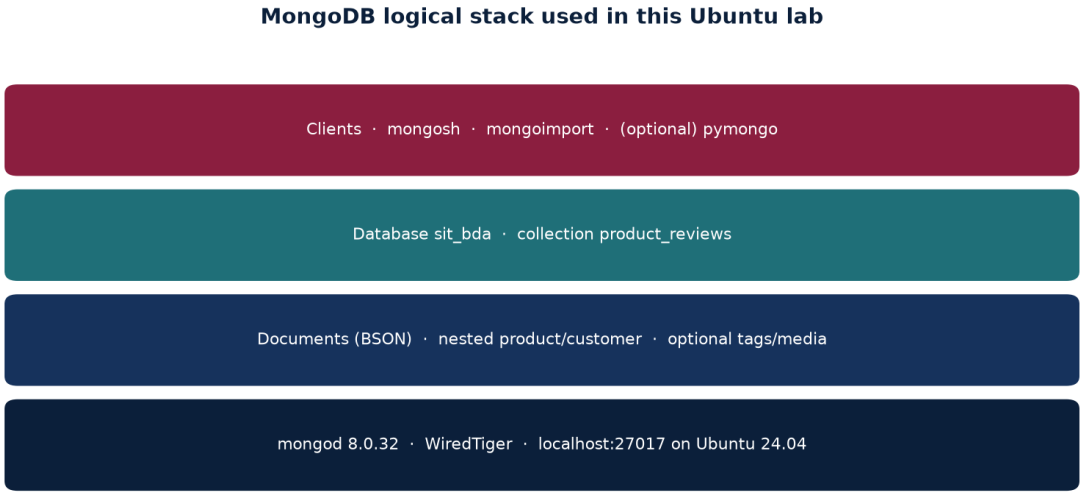


Figure 2. mongod → database sit_bda → collection product_reviews → BSON documents.

6.2 CAP theorem (viva)

Brewer’s CAP theorem says a distributed store cannot simultaneously provide consistency, availability and partition tolerance. Network partitions happen, so the practical choice is CP versus AP. A MongoDB replica set defaults toward CP: a primary takes writes; if the primary is partitioned away, a new primary is elected and writes wait. Read concern / write concern can be

relaxed toward availability. This Ubuntu lab is a single-node mongod, so CAP is discussed theoretically rather than demonstrated with a replica-set partition.

6.3 Loose JSON Schema

MongoDB can run with no validator at all. For the lab a loose $jsonSchema was attached: review_id, product, rating and body are required; `additionalProperties: true` lets every other key through. That is the opposite of a relational CREATE TABLE that lists every column up front. It still blocks obviously broken inserts (no rating, empty body) — a compromise used in production lakes as well.

7. Dataset and schema-less design

The corpus is a synthetic but realistic e-commerce review dump: nested product (sku, name, brand, category), nested customer (handle, city, sometimes verified_purchase), free-text body, and optional extras. A Python generator with a fixed seed (the PRN) wrote JSON Lines so `mongoimport` can stream one document per line.

Top-level field	Documents	Presence
review_id	300	100%
captured_at	300	100%
channel	300	100%
product	300	100%
customer	300	100%
rating	300	100%
body	300	100%
language	300	100%
tags	237	79%
title	216	72%
helpful_votes	132	44%
media	60	20%
raw_payload_excerpt	44	15%
seller_reply	30	10%
ingest_meta	27	9%
return_reason	5	2%

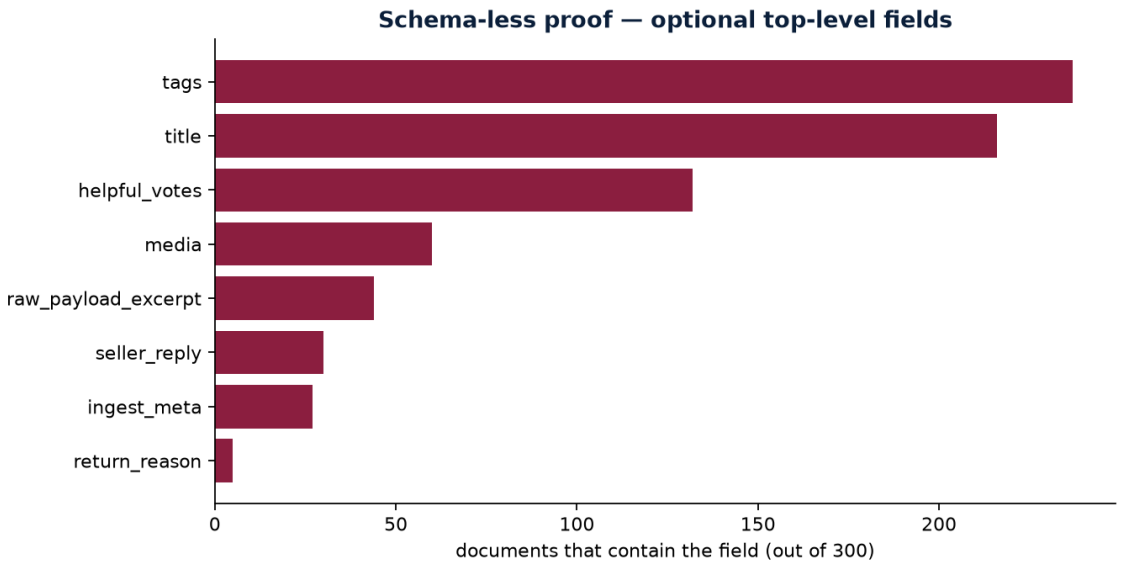


Figure 3. Optional keys (tags, title, media, seller_reply, return_reason) are the unstructured part.

If this were PostgreSQL, every optional field would be a nullable column or a JSONB sidecar. Here they simply appear on some documents. REV-0001 even carries return_reason and media while REV-0002 carries ingest_meta and tags — two neighbouring rows that would not share a column set.

8. Implementation procedure (Ubuntu)

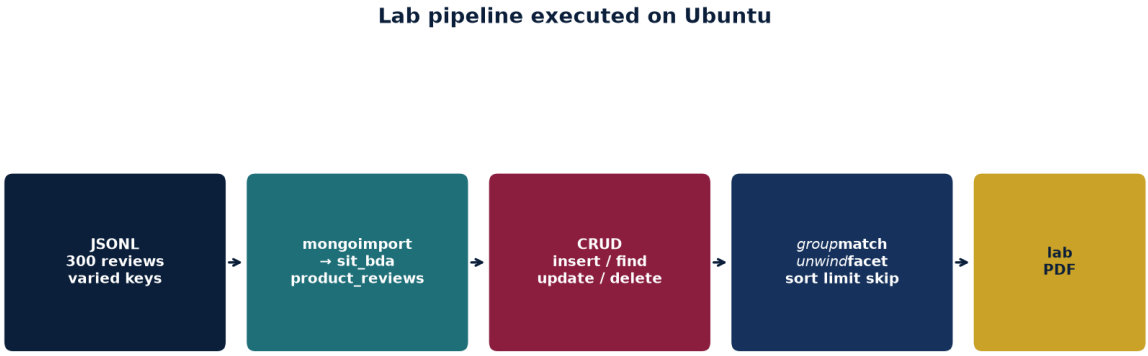


Figure 4. End-to-end lab pipeline actually executed.

Step	What was done
1. Install	Official MongoDB 8.0 APT repo on Ubuntu noble; packages mongodb-org, mongosh, mongoimport.
2. Start mongod	Forked mongod on 127.0.0.1:27017 (no systemd in this lab VM). ping {ok:1}.
3. Generate JSONL	python3 scripts/generate_dataset.py → 300 documents.
4. Create collection	mongosh scripts/setup.js — sit_bda.product_reviews + loose validator.
5. Import	mongoimport --db sit_bda --collection product_reviews --file dataset/product_reviews.jsonl
6. Query	mongosh --file scripts/queries.js — CRUD, count/sort/limit/skip, aggregations.

Re-running the lab is bash `scripts/run_lab.sh` with mongod already listening. Output transcripts are saved under `results/`.

9. Source listings

9.1 Collection setup (scripts/setup.js)

The validator requires four fields only. additionalProperties remains true so unstructured extras survive.

```
// Create the sit_bda.product_reviews collection with a *loose* JSON Schema.
// additionalProperties: true keeps the store schema-less (unstructured / semi-structured).
const DB_NAME = "sit_bda";
const COLL = "product_reviews";

db = db.getSiblingDB(DB_NAME);
db[COLL].drop();
const res = db.createCollection(COLL, {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["review_id", "product", "rating", "body"],
      properties: {
        review_id: { bsonType: "string" },
        rating: { bsonType: ["int", "double", "long"], minimum: 1, maximum: 5 },
        product: { bsonType: "object" },
        body: { bsonType: "string" },
      },
      additionalProperties: true,
    },
  },
});
printjson({
  database: DB_NAME,
  collection: COLL,
  created: res,
  note: "JSON Schema requires four fields only; extra nested keys are allowed.",
});
printjson(db.getCollectionInfos({ name: COLL }));
```

9.2 Query script — CRUD and aggregation (excerpt from scripts/queries.js)

```
banner("5. CRUD — insertOne with extra nested junk a RDBMS would reject");
const insertRes = db[COLL].insertOne({
  review_id: "REV-LAB-INSERT",
  captured_at: "2026-09-13T20:15:00+05:30",
  channel: "lab-terminal",
  product: {
    sku: "LAB-0001",
    name: "MongoDB lab checklist",
    brand: "SIT-AIML",
    category: "books",
    extra_unstructured: { printed: false, url: "file://queries.js" },
  },
  customer: { handle: "sukhada", city: "Pune", verified_purchase: true, prn: "24070126505" },
  rating: 5,
  title: "Inserted during the Ubuntu lab",
  body: "This document was inserted with mongosh insertOne to demonstrate CRUD on unstructured JSON.",
  tags: ["lab", "nosql", "mongodb", "batch-a3"],
  language: "en",
  viva_note: "No ALTER TABLE was required to add viva_note, tags or nested extra_unstructured.",
});
printjson({ acknowledged: insertRes.acknowledged, insertedId: insertRes.insertedId });
print("count after insert = " + db[COLL].countDocuments());

banner("6. CRUD — find (filter + projection), like SQL WHERE + SELECT cols");
print("electronics, rating >= 4, first 3 (projection hides _id and body):");
printjson(
  db[COLL]
    .find(
      { "product.category": "electronics", rating: { $gte: 4 } },
      { _id: 0, review_id: 1, rating: 1, title: 1, "product.name": 1, "customer.city": 1 }
    )
    .limit(3)
    .toArray()
);

banner("7. CRUD — updateOne");
const upd = db[COLL].updateOne(
  { review_id: "REV-LAB-INSERT" },
  { $set: { lab_verified: true, faculty_flag: "ready-for-viva" }, $inc: { helpful_votes: 1 } }
);
printjson(upd);
printjson(db[COLL].findOne({ review_id: "REV-LAB-INSERT" }, { _id: 0, body: 0 }));

banner("8. CRUD — deleteOne");
```

```
const del = db[COLL].deleteOne({ review_id: "REV-LAB-INSERT" });
printjson(del);
print("count after delete = " + db[COLL].countDocuments());


// --- aggregation ---

banner("10. aggregation — reviews and average rating by category");
const byCat = db[COLL]
  .aggregate([
    {
      $group: {
        _id: "$product.category",
        n_reviews: { $sum: 1 },
        avg_rating: { $avg: "$rating" },
        min_rating: { $min: "$rating" },
        max_rating: { $max: "$rating" },
      },
    },
    { $sort: { n_reviews: -1 } },
    {
      $project: {
        _id: 0,
        category: "$_id",
        n_reviews: 1,
        avg_rating: { $round: ["$avg_rating", 2] },
        min_rating: 1,
        max_rating: 1,
      },
    },
  ])
  .toArray();
printjson(byCat);
print("compact:");
compactTable(
  byCat.map((r) => [r.category, r.n_reviews, r.avg_rating, r.min_rating, r.max_rating]),
  ["category", "n", "avg", "min", "max"]
);
```

The full mongosh file (identity, schema-less $objectToArray, Pune $match, $unwind tags, $facet) is submitted alongside this PDF as scripts/queries.js.

10. Execution screenshots

All terminals below are painted from the real stdout/stderr of the Ubuntu session (MongoDB 8.0.32). Host prompt is labelled as the student lab machine.

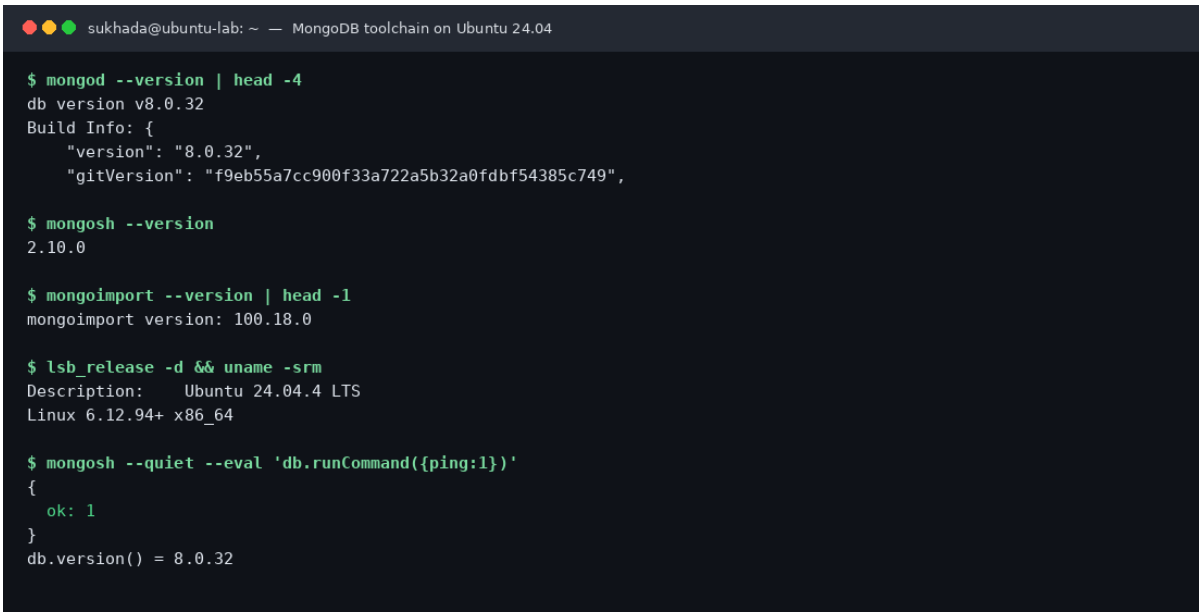


Figure 5. mongod 8.0.32, mongosh 2.10.0, mongoimport 100.18.0, Ubuntu 24.04, ping ok:1.

sukhada@ubuntu-lab: ~/nosql — schema-less review JSON

```
$ python3 - <<'PY'   # pretty-print first two JSONL records
--- document 1 ---
{
  "review_id": "REV-0001",
  "captured_at": "2026-06-28T03:14:00+05:30",
  "channel": "web",
  "product": {
    "sku": "CL-7720",
    "name": "Running shorts",
    "brand": "PaceForm",
    "category": "clothing"
  },
  "customer": {
    "handle": "ananya_fit",
    "city": "Bengaluru"
  },
  "rating": 1,
  "body": "Sunscreen pills on the nose. Fine for office AC, not for a noon cricket match.",
  "language": "mr",
  "title": "Exceeded expectations",
  "helpful_votes": 14,
  "media": [
    {
      "type": "image",
      "n": 3
    }
  ],
  "return_reason": "late-delivery"
}
```

Figure 6. One unstructured review: nested product/customer, optional title, media, return_reason.

sukhada@ubuntu-lab: ~/nosql — createCollection + loose JSON Schema

```
$ mongosh mongodb://127.0.0.1:27017 --file scripts/setup.js
{
  database: 'sit_bda',
  collection: 'product_reviews',
  created: {
    ok: 1
  },
  note: 'JSON Schema requires four fields only; extra nested keys are allowed.'
}
  validator.$jsonSchema.required: [review_id, product, rating, body]
  additionalProperties: true
```

Figure 7. createCollection with JSON Schema — additionalProperties: true.

sukhada@ubuntu-lab: ~/nosql — mongoimport JSONL

```
$ mongoimport --uri mongodb://127.0.0.1:27017 --db sit_bda --collection product_reviews --file
dataset/product_reviews.jsonl
2026-09-13T20:11:03.948+0000    connected to: mongodb://127.0.0.1:27017
2026-09-13T20:11:03.955+0000    300 document(s) imported successfully. 0 document(s) failed to
import.
```

Figure 8. mongoimport: 300 document(s) imported successfully, 0 failed.

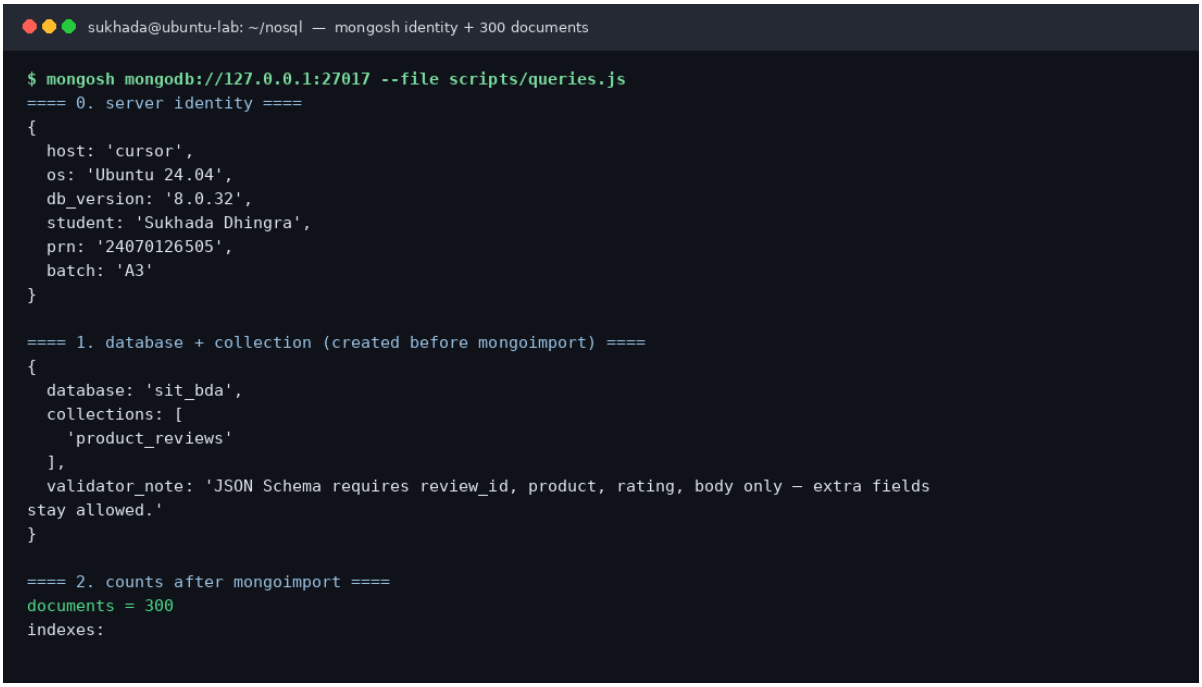


Figure 9. mongosh session stamped with student name, PRN 24070126505, Batch A3; 300 docs.

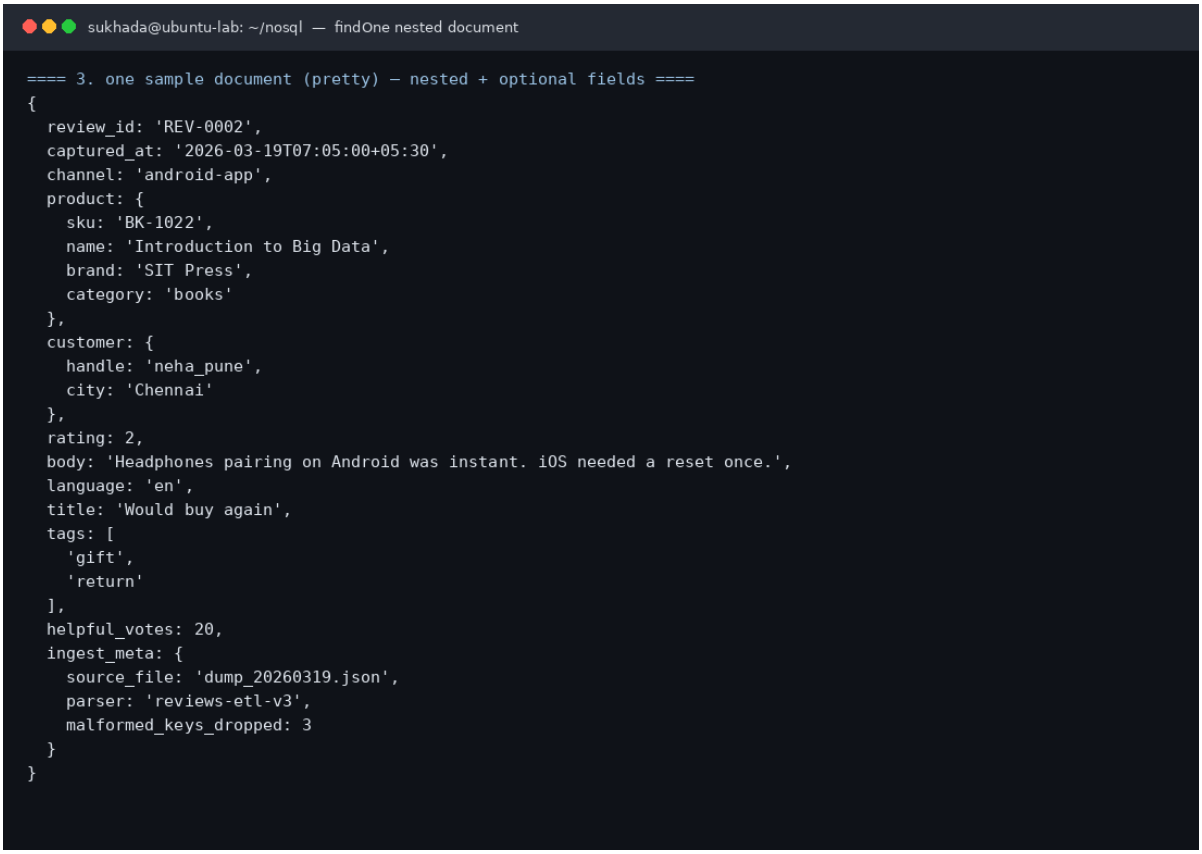


Figure 10. findOne() of a nested BSON document after import.

```
sukhada@ubuntu-lab: ~/nosql — optional fields (schema-less store)

$  # field presence after $objectToArray / $group
compact:
field                n    present
-------------------  ---  -------
body                 300  100%
captured_at          300  100%
_id                  300  100%
rating               300  100%
language             300  100%
channel              300  100%
customer             300  100%
product              300  100%
review_id            300  100%
tags                 237  79%
title                216  72%
helpful_votes        132  44%
media                60   20%
raw_payload_excerpt  44   15%
seller_reply         30   10%
ingest_meta          27   9%
return_reason        5    2%
```

Figure 11. Field-presence aggregation — tags 79%, title 72%, return_reason 2%.

```
sukhada@ubuntu-lab: ~/nosql — insertOne / updateOne / deleteOne

==== 5. CRUD — insertOne with extra nested junk a RDBMS would reject ====
{
  acknowledged: true,
  insertedId: ObjectId('6aa70358cf0f2487af10f604')
}
count after insert = 301


==== 7. CRUD — updateOne ====
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
==== 8. CRUD — deleteOne ====
{
  acknowledged: true,
  deletedCount: 1
}
count after delete = 300
```

Figure 12. insertOne → 301 documents; updateOne matched; deleteOne → 300 again.

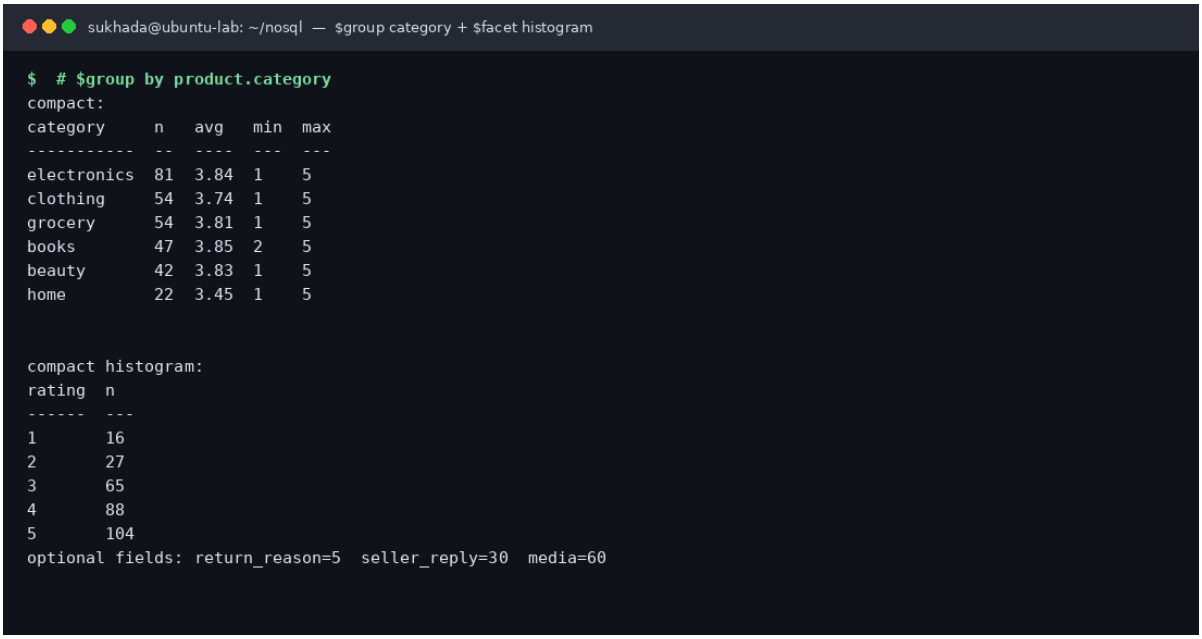


Figure 13. $group by category (electronics 81 @ 3.84) and $facet rating histogram.

11. Results and analysis

11.1 Import and CRUD

Check	Result	Meaning
mongoimport	300 imported / 0 failed	JSONL parsed as one BSON document per line
countDocuments after import	300	Collection populated
insertOne REV-LAB-INSERT	301	Schema accepted extra nested keys (viva_note, tags)
updateOne $set / $inc	matchedCount 1	New fields lab_verified, faculty_flag added in place
deleteOne	300	Lab insert removed; corpus restored
count rating == 1	16	Low-star slice for quality monitoring

11.2 Aggregation by category

Category	n	avg rating	min	max
electronics	81	3.84	1	5
clothing	54	3.74	1	5
grocery	54	3.81	1	5
books	47	3.85	2	5
beauty	42	3.83	1	5
home	22	3.45	1	5

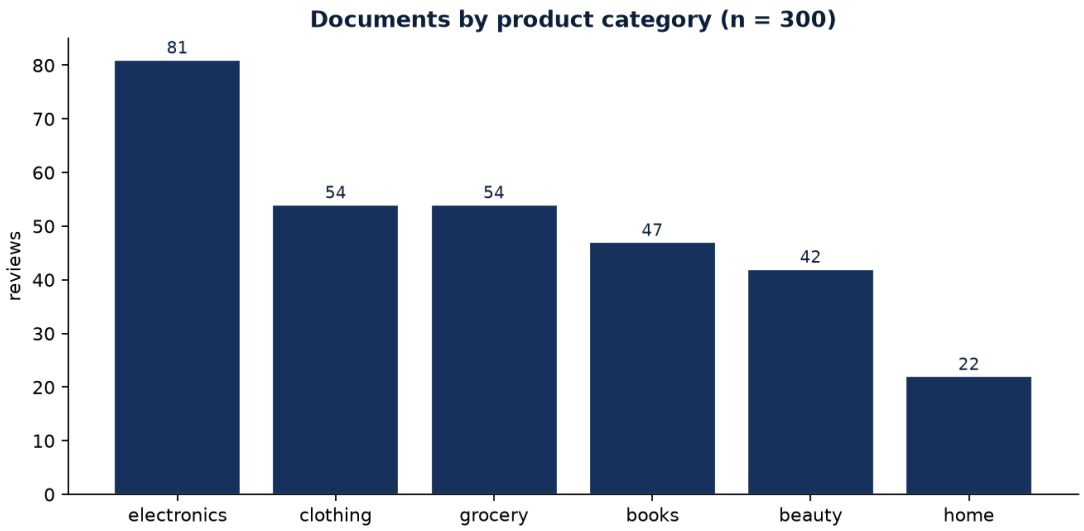


Figure 14. Review volume by category — electronics is the largest slice of the dump.

Books have the highest mean rating (3.85) and no 1-star rows in this sample (min 2). Home is smallest and weakest (22 reviews, 3.45). A relational GROUP BY would need a reviews table plus a products table and a join; MongoDB grouped on the embedded path product.category.

11.3 Rating histogram and optional-field analytics

Star rating	Count	Share of 300
1	16	5.3%
2	27	9.0%
3	65	21.7%
4	88	29.3%
5	104	34.7%

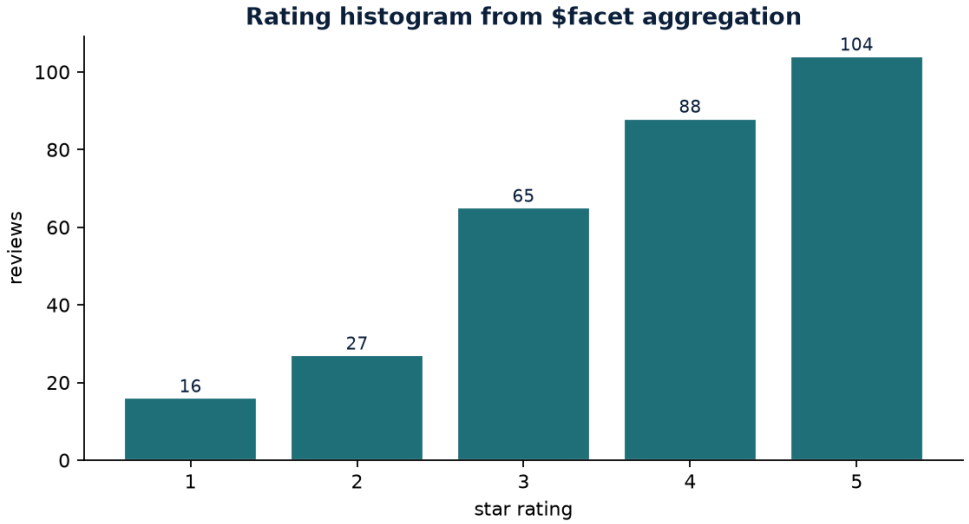


Figure 15. $facet histogram — the dump is skewed toward 4–5 star reviews.

$facet also counted documents that only some reviews carry: return_reason = 5, seller_reply = 30, media = 60. Those counts are exactly the schema-less fields from §7. $unwind on tags ranked topics (loud 42, fading 39, fit 37, …, ubuntu 35) — array analytics that would need a bridge table in SQL.

11.4 Pune slice

A $match on customer.city = "Pune" followed by $group on product.brand is the NoSQL form of SELECT brand, COUNT(*), AVG(rating) WHERE city = 'Pune' GROUP BY brand. Top Pune brands in this run were SonicPulse, IndigoYard and VoltNest (6 reviews each). Clickora had the best Pune mean among the displayed six (4.25).

12. Observations, viva notes and limitations

12.1 Observations

- JSONL (one object per line) is the correct import format; a giant JSON array needs --jsonArray.
- Missing fields are not errors. find({title: {$exists: false}}) is how you hunt incomplete records.
- Embedded product.category avoids a join for the main grouping query in this lab.
- insertOne accepted viva_note and extra_unstructured without any DDL — the unstructured lesson.
- deleteOne of the lab document restored the 300-row corpus; labs should always clean up.
- Aggregation pipelines are ordered stages, not a single SQL text blob — easier to extend.

12.2 Likely viva questions (with short answers)

Question	Model answer
What is unstructured data here?	JSON reviews with nested objects, arrays and keys that appear only on some documents.
Why not MySQL?	Optional nested fields would force NULLs, JSON columns or frequent ALTER TABLE. MongoDB stores the native shape.
Document vs key–value?	Key–value is opaque get/put. Document stores index fields inside the value and run aggregations.
CAP for this lab?	Single-node mongod; theoretically a replica set defaults CP (primary writes, election on partition).
BSON vs JSON?	BSON is MongoDB’s typed binary encoding (dates, ObjectId, int vs double) of a JSON-like document.
What does $unwind do?	It emits one document per array element so $group can count tags.
mongoimport vs insertMany?	mongoimport is the CLI bulk loader from files; insertMany is the driver/API equivalent.
additionalProperties: true?	JSON Schema will not reject unknown keys — required for unstructured ingest.

12.3 Limitations

The corpus is 300 documents so every counter fits on a screenshot; a production lake would be millions of events and would use sharded clusters, hashed shard keys and indexes on product.sku and customer.city. This VM has no systemd, so mongod was forked by hand. Replica-set failover and authentication were not enabled. Text search used a regex on body rather than a text index. Those are acceptable limits for a first NoSQL ingest lab.

13. Conclusion

This laboratory exercise for Big Data Analytics showed how unstructured JSON is stored and queried in a NoSQL document database. I, Sukhada Dhingra (PRN 24070126505, Batch A3), installed MongoDB 8.0.32 on Ubuntu 24.04, generated 300 schema-less product reviews,

imported them with zero failures, and executed CRUD plus aggregation pipelines in mongosh. Field-presence analysis confirmed the unstructured claim (return_reason on only 5/300 documents). Category aggregation and a rating histogram produced analytics that would otherwise need joins and sparse columns. Screenshots of version, import SUCCESS, CRUD counts and compact aggregation tables are attached as evidence. The same ingest-then-query pattern is how click logs, IoT payloads and social feeds enter a modern data platform.

14. References

[1] MongoDB Inc. (2026). Install MongoDB Community Edition on Ubuntu. https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-ubuntu/

[2] MongoDB Inc. mongoimport, mongosh, Aggregation Pipeline, $jsonSchema documentation.

[3] Sadalage, P. J. and Fowler, M. NoSQL Distilled. Addison-Wesley.

[4] Brewer, E. (2012). CAP twelve years later: How the “rules” have changed. IEEE Computer.

[5] Dean, J. and Ghemawat, S. (2004). MapReduce: Simplified Data Processing on Large Clusters. OSDI. (Big Data processing context.)

[6] Course notes: Big Data Analytics — NoSQL databases, schema-less models and MongoDB laboratory.

— End of report —
Submitted by Sukhada Dhingra · PRN 24070126505 · Batch A3